CTF 過程記錄

1. VPN 連線成功:使用 OpenVPN 搭配 lab.ovpn 設定檔並輸入學號驗證，成功建立連線。

```
┌──(kali㉿kali)-[~]
└─$ sudo openvpn --config lab.ovpn
2026-03-17 02:52:13 DEPRECATED: --persist-key option ignored. Keys are now always persisted across restarts
.
2026-03-17 02:52:13 Note: --cipher is not set. OpenVPN versions before 2.5 defaulted to BF-CBC as fallback
when cipher negotiation failed in this case. If you need this fallback please add '--data-ciphers-fallback
BF-CBC' to your configuration and/or add BF-CBC to --data-ciphers. E.g. --data-ciphers DEFAULT:BF-CBC
2026-03-17 02:52:13 OpenVPN 2.7_rc2 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PK
TINFO] [AEAD] [DCO]
2026-03-17 02:52:13 library versions: OpenSSL 3.5.4 30 Sep 2025, LZO 2.10
2026-03-17 02:52:13 DCO version: 6.16.8+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1 (2025-09-24)
Enter Auth Username: 412630369
Enter Auth Password: ••••••••••
2026-03-17 02:52:20 TCP/UDP: Preserving recently used remote address: [AF_INET]163.13.203.1:1195
2026-03-17 02:52:20 Attempting to establish TCP connection with [AF_INET]163.13.203.1:1195
2026-03-17 02:52:20 TCP connection established with [AF_INET]163.13.203.1:1195
2026-03-17 02:52:20 TCPv4_CLIENT link local: (not bound)
2026-03-17 02:52:20 TCPv4_CLIENT link remote: [AF_INET]163.13.203.1:1195
2026-03-17 02:52:20 WARNING: this configuration may cache passwords in memory -- use the auth-nocache optio
n to prevent this
2026-03-17 02:52:20 [TKUIMWD_CTF] Peer Connection Initiated with [AF_INET]163.13.203.1:1195
2026-03-17 02:52:21 sitnl_send: rtnl: generic error (-17): File exists
2026-03-17 02:52:21 DCO device tun1 opened
2026-03-17 02:52:21 ovpn-dco device [tun1] opened
2026-03-17 02:52:21 net_iface_mtu_set: mtu 1500 for tun1
2026-03-17 02:52:21 net_iface_up: set tun1 up
2026-03-17 02:52:21 net_addr_ptp_v4_add: 172.16.101.10 peer 172.16.101.9 dev tun1
2026-03-17 02:52:21 Initialization Sequence Completed
```

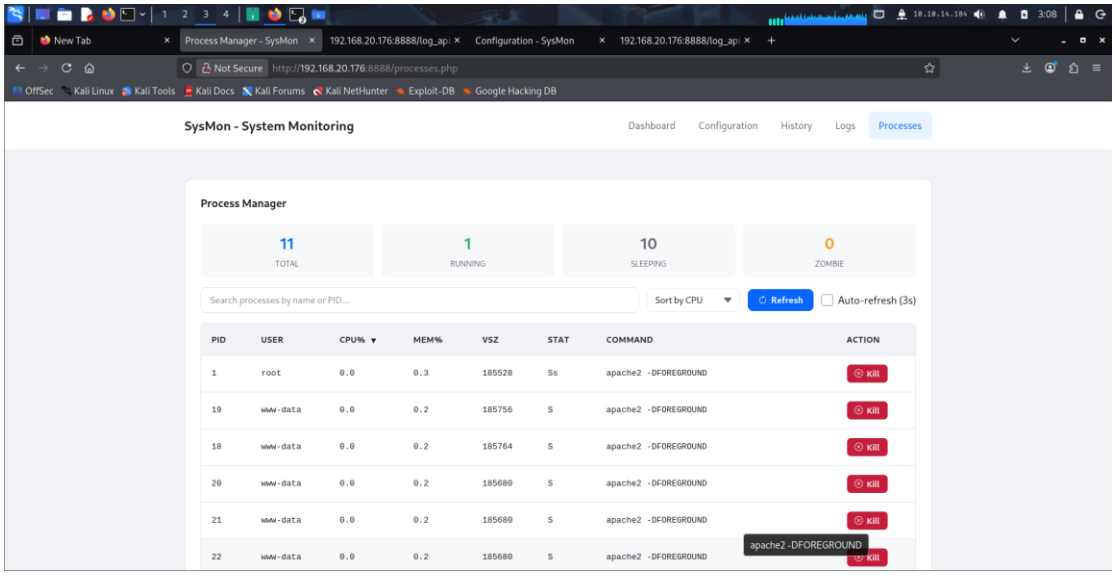
2. 確認目標連線:透過 ping 指令測試，確認 Kali 與目標伺服器 192.168.20.176 通訊正常，網路已連通。

```
┌──(kali㉿kali)-[~]
└─$ ping -c 4 192.168.20.176
PING 192.168.20.176 (192.168.20.176) 56(84) bytes of data.
64 bytes from 192.168.20.176: icmp_seq=1 ttl=63 time=17.2 ms
64 bytes from 192.168.20.176: icmp_seq=2 ttl=63 time=18.7 ms
64 bytes from 192.168.20.176: icmp_seq=3 ttl=63 time=17.8 ms
64 bytes from 192.168.20.176: icmp_seq=4 ttl=63 time=17.0 ms

--- 192.168.20.176 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3075ms
rtt min/avg/max/mdev = 16.965/17.658/18.740/0.687 ms
```

3. 取得白箱原始碼:解壓縮 sysmon-web.zip 檔，列出網頁目錄架構，準備對 PHP 檔案進行漏洞分析。

```
┌──(kali㉿kali)-[~]
└─$ cd ~/Downloads/sysmon-web
ls -R
.:
docker-compose.yml  docker-entrypoint.sh  Dockerfile  index.php  README.md  www

./www:
config_action.php  generate_logs.php  log_api.php     processes.php    style.css
config_api.php     history.php        logs.php        reset_config.php
config.php         index.php          process_api.php status.php
```

4. 訪問監控面板:瀏覽器連上目標網頁，這是一個顯示 CPU 與記憶體狀態的系統監控平台。



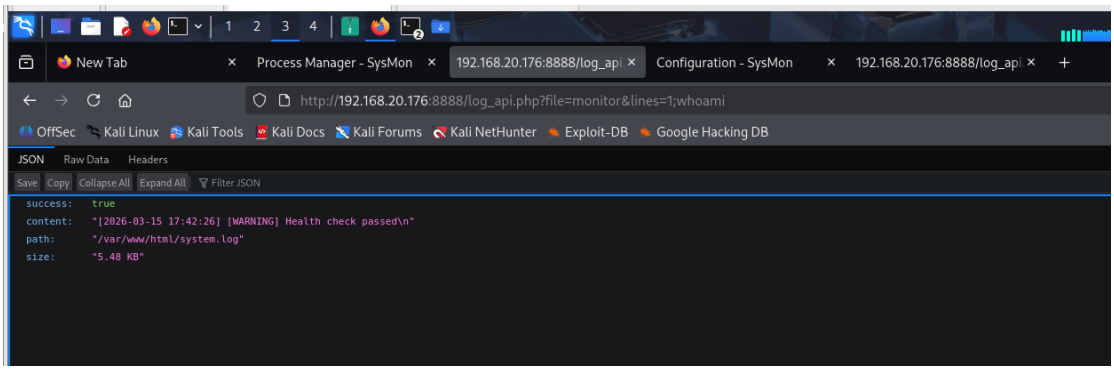
5. 分析進程管理代碼:查看 process_api.php，發現程式會執行 ps 與 kill 指令，但 PID 有經過數字過濾。

```
┌──(kali㉿kali)-[~/Downloads/sysmon-web]
└─$ cat ~/Downloads/sysmon-web/www/process_api.php
<?php
session_start();
header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = $_POST['action'] ?? '';
    $pid = intval($_POST['pid'] ?? 0);

    if ($action === 'kill' && $pid > 1) {
        $result = shell_exec("kill -9 $pid 2>&1");

        if (strpos($result, 'No such process') !== false) {
            echo json_encode(['success' => false, 'error' => 'Process not found']);
        } else {
            echo json_encode(['success' => true]);
```

6. 分析日誌讀取代碼:查看 log_api.php，發現日誌路徑被寫死，但支援讀取 Web 目錄下的 system.log。

```
┌──(kali㉿kali)-[~/Downloads/sysmon-web]
└─$ cat ~/Downloads/sysmon-web/www/log_api.php
<?php
session_start();
header('Content-Type: application/json');

$logFiles = [
    'syslog' => '/var/log/syslog',
    'auth' => '/var/log/auth.log',
    'apache' => '/var/log/apache2/access.log',
    'apache-error' => '/var/log/apache2/error.log',
    'monitor' => '/var/www/html/system.log'
];
```

7. 嘗試測試指令注入:在網址列嘗試在參數後加入 whoami 指令，初步測試伺服器對特殊字元的反應。



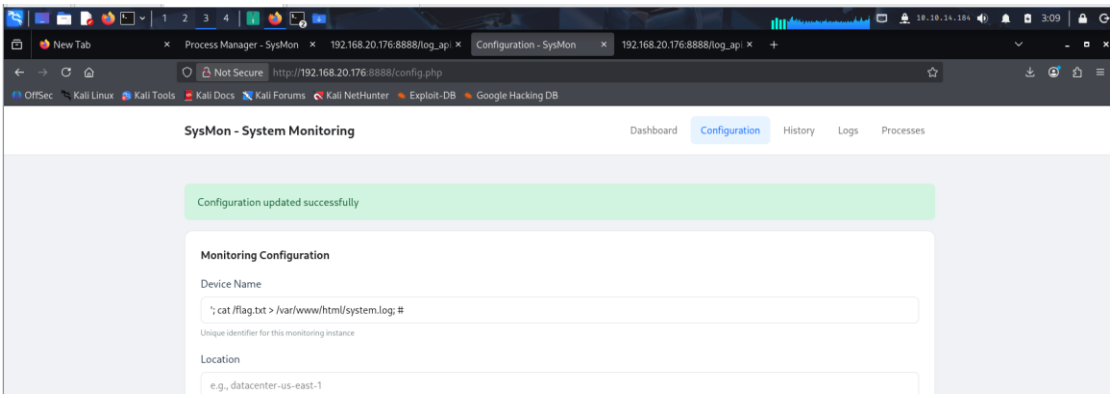
8. 鎖定關鍵漏洞點:查看 config_action.php，發現設定值會直接串接到 exec() 系統指令中執行，且過濾不嚴。

```
┌──(kali㉿kali)-[~/Downloads/sysmon-web]
└─$ cat ~/Downloads/sysmon-web/www/config_action.php
<?php
session_start();
header('Content-Type: application/json; charset=UTF-8');

function applyConfiguration() {
    $config_file = '/var/www/html/current_config.json';
    $config = array();

    if (file_exists($config_file)) {
        $config = json_decode(file_get_contents($config_file), true);
```

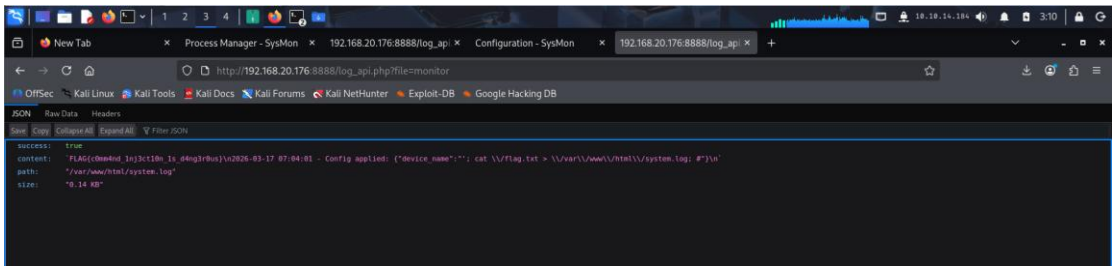
9. 定位 Flag 存放位置:使用 grep 指令掃描原始碼，發現真正的 Flag 檔案存放於伺服器根目錄的 /flag.txt。

```
┌──(kali㉿kali)-[~/Downloads/sysmon-web]
└─$ grep -r "flag" ~/Downloads/sysmon-web/
/home/kali/Downloads/sysmon-web/Dockerfile:RUN echo "FLAG{EXAMPLE_FLAG}" > /flag.txt && \
/home/kali/Downloads/sysmon-web/Dockerfile:    chmod 644 /flag.txt
```

10. 執行指令注入攻擊:在設定頁面輸入 Payload，利用單引號逃逸並強制執行 cat /flag.txt 將 Flag 寫入日誌。



11. 成功讀取 Flag 內容:重新訪問日誌介面，在 system.log 的回傳內容中，成功看到 Flag 字串。



FLAG : c0mm4nd_1nj3ct10n_1s_d4ng3r0us